

Play Framework Grundlagen

Play Framework: Grundlagen

Gliederung



- 1. Was ist Play?
- 2. Projektaufbau & Architektur
- 3. Routing
- 4. Actions & Results
- 5. Play JSON
- 6. Dependency Injection
- 7. Schichten am Beispiel Wetter
- 8. Funktionale Fehlerbehandlung (EitherT)
- 9. Übung: Todo-Feature in 10 Schritten

Was ist Play?

Was ist Play?

Play Framework



- **The High Velocity Web Framework for Java and Scala**
- Web-Framework für moderne, skalierbare Anwendungen
- Unterstützung für reaktive Anwendungen nach dem **Reactive Manifesto**
- Baut auf **Pekko** (seit Version 3, davor Akka, mehr dazu in einer späteren Vorlesung) auf
 - Aktorsystem im Hintergrund für hohe Parallelität in der Request-Verarbeitung
- Stateless, non-blocking, asynchron by default

Was ist Play?

Warum Play für FP?

- Alles kommt hier zusammen:
 - **Future** (VL05): asynchrone Request-Verarbeitung
 - **Either / EitherT** (VL03/VL06): funktionale Fehlerbehandlung
 - **Typklassen** (VL04/VL06): JSON-Serialisierung mit Reads/Writes
 - **cats** (VL06): EitherT, traverse, leftWiden im Service
- Play ist ein konkreter Anwendungsfall für funktionale Programmierung

Projektaufbau & Architektur

Projektaufbau & Architektur

Projektstruktur



- **app/** — Quellcode, organisiert nach **Features** (nicht nach Typen)
- **conf/** — `routes` und `application.conf`
- **project/** — sbt-Plugins (`plugins.sbt`) und sbt-Version
- **app/Module.scala** — Guice-Modul (oft leer dank `@ImplementedBy`)

app/

common/models/errors/ ApplicationError, BadRequest

weather/

 application/ WeatherController

 domain/ WeatherService, models, errors

 infrastructure/ Clients, Repositories

Projektaufbau & Architektur

Vertical Slices & Schichten



- Jedes Feature ist ein **vertikaler Schnitt** in drei Schichten:
- **application:** Controller, nimmt Requests entgegen, gibt Results zurück
- **domain:** Geschäftslogik: Service, Modelle
- **infrastructure:** HTTP-Clients, DB (hinter Trait)
- Vorteil: hohe Kohäsion pro Feature, klare Abhängigkeitsrichtung nach innen

- **Anfrage:** Client → Router → Controller (application) → Service (domain)
- **Antwort:** Service liefert Ergebnis, Controller mappt auf eine HTTP-Response
- Beispiel Wetter:
 - `POST /v1/weather` → `WeatherController` → `WeatherService` → `WeatherClient` → Open-Meteo

Routing

Routing

Die routes-Datei

- Zentrale Routing-Datei: **conf/routes**
- Syntax: ``HTTP_METHOD URL_PFAD Controller.Action``
- Parameter werden **automatisch typisiert** gebunden (Long, Int, String, UUID)

weather-Feature (bereits vorhanden)

```
POST /v1/weather      weather.application.WeatherController.getWeatherForCity()
```

todo-Feature (Übung)

```
GET /v1/todos/:id     todo.application.TODOController.getById(id: Long)
```

...

Actions & Results

- Eine **Action** ist eine Funktion von Request auf Result
- Play stellt den **Action-Builder** bereit:
- Play kümmert sich intern um die Ausführung der Futures

```
// Asynchrone Action – gibt Future[Result] zurück
def getAll = Action.async { request =>
  todoService.findAll.value.map(_ => Ok("..."))
}
```

Actions & Results

Results

- Play liefert vordefinierte Result-Objekte für HTTP-Antworten:
- Results können Content tragen: Text, JSON, Dateien
- Framework setzt passende Content-Types und Status-Codes

<code>Ok(Json.toJson(todo))</code>	<code>// 200 OK mit JSON</code>
<code>Created(Json.toJson(todo))</code>	<code>// 201 Created</code>
<code>NoContent</code>	<code>// 204 No Content</code>
<code>BadRequest(error.message)</code>	<code>// 400 BadRequest</code>
<code>NotFound(error.message)</code>	<code>// 404 NotFound</code>
<code>InternalServerError("InternalServerError")</code>	<code>// 500 InternalServerError</code>
<code>BadGateway(error.message)</code>	<code>// 502 BadGateway</code>

Play JSON

- **Play-JSON**: Scala-typische API für JSON (Case Classes, Option, ...)
- **Format** automatisch per Makro ableiten — als `given` im Companion Object
- `Json.format[T]` erzeugt **Reads** (JSON→Scala) und **Writes** (Scala→JSON)

```
import play.api.libs.json.{Format, Json}
```

```
case class City(name: String, lat: Double, lon: Double)
```

```
object City {  
  given format: Format[City] = Json.format[City]  
}
```


- Objekt zu JSON konvertieren und als Response senden
- `Json.toJson` nutzt das implizite **Writes/Format** des Typs
- Play setzt automatisch den Content-Type `application/json`

```
def getById(id: Long) = Action.async {  
  todoService.findById(id).value.map {  
    case Right(todo) => Ok(Json.toJson(todo))  
    case Left(error) => NotFound(error.message)  
  }  
}
```

- **Typsicherer** Body-Parser `parse.json[T]` — direkt zur Case Class:
- `request.body` ist bereits ein `City` — kein manuelles `validate` nötig
- Voraussetzung: `given Format[City]` im Scope

```
def getWeatherForCity() = Action.async(parse.json[City]) { request =>
  weatherService.getWeatherForCity(request.body).value.map {
    case Right(weather) => Ok(s"$weather")
    case Left(error)    => InternalServerError(s"Error: $error")
  }
}
```

Dependency Injection

Dependency Injection

Warum Dependency Injection?



- **Lose Kopplung:** Schichten kennen nur Traits, nicht die Implementierung
- **Testbarkeit:** im Test eine Mock-Implementierung einbinden
- **Austauschbarkeit:** z.B. InMemory ↔ DB ohne Änderung an Controller/Service
- Play verwendet standardmäßig **Guice** für DI

@Singleton

```
class WeatherController @Inject() (  
    cc: ControllerComponents,  
    weatherService: WeatherService // nur das Trait!  
) (using ExecutionContext) extends AbstractController(cc)
```

Dependency Injection

Bindings mit **@ImplementedBy**



- Trait und Implementierung verknüpfen, **ohne** Eintrag in Module.scala:
- Guice bindet automatisch `WeatherService` → `WeatherServiceImpl`
- Module.scala bleibt leer (nur bei mehreren/umgebungsabhängigen Impls nötig)

```
@ImplementedBy(classOf[WeatherServiceImpl])  
trait WeatherService {  
    def getWeatherForCity(city: City): EitherT[Future, WeatherDataError, WeatherData]  
}
```

Dependency Injection

Under the hood



- **Beim Start liest Guice alle Bindings:** aus `@ImplementedBy`, aus `Module.scala` und „just-in-time“ aus den Klassen selbst
- Per **Reflection** findet Guice den mit `@Inject` markierten Konstruktor und dessen Parameter-Typen
- **Daraus baut Guice einen Abhängigkeitsgraphen** → zu jedem Typ ist hinterlegt, wie eine Instanz erzeugt wird
- Wird ein Objekt angefordert, löst Guice rekursiv zuerst dessen Abhängigkeiten auf (erst Repository, dann Service, dann Controller) und ruft danach den Konstruktor auf

Fehlende oder zyklische Abhängigkeiten erkennt Guice schon beim Start →
die App startet gar nicht erst (fail-fast statt Laufzeitfehler)

Schichten am Beispiel Wetter

Schichten am Beispiel Wetter

Das weather-Feature im Überblick



- Drei Dateien, drei Schichten — als **Vorlage** für eure Übung:
- **application/** `WeatherController` — HTTP, `POST /v1/weather`
- **domain/** `WeatherService` (+ Impl) — Logik, Rückgabe `EitherT[Future, ...]`
- **infrastructure/** `WeatherClient` (+ `OpenMeteoWeatherClient`) — HTTP-Call
- Modelle & Fehler in `domain/models` bzw. `common/models/errors`

Schichten am Beispiel Wetter

infrastructure: WeatherClient

- Trait beschreibt den Zugriff, `@ImplementedBy` bindet die Implementierung:

```
@ImplementedBy(classOf[OpenMeteoWeatherClient])
private[weather] trait WeatherClient {
  def currentWeather(name: String, lat: Double, lon: Double)
    : EitherT[Future, WeatherDataError, WeatherData]
}

private[infrastructure] class OpenMeteoWeatherClient @Inject() ()(
  using ec: ExecutionContext
) extends WeatherClient {
  // HTTP-Call an Open-Meteo, JSON mit playJson parsen
}
```

Schichten am Beispiel Wetter

domain: **WeatherService**

- Businesslogik, unabhängig vom Transport — gibt `EitherT` zurück:

```
@ImplementedBy(classOf[WeatherServiceImpl])  
trait WeatherService {  
  def getWeatherForCity(city: City): EitherT[Future, WeatherDataError, WeatherData]  
}
```

Schichten am Beispiel Wetter

application: WeatherController

- Nur HTTP: Body parsen, Service rufen, Ergebnis auf Result mappen:

@Singleton

```
class WeatherController @Inject() (  
    cc: ControllerComponents,  
    weatherService: WeatherService  
) (using ExecutionContext) extends AbstractController(cc) {  
  
    def getWeatherForCity() = Action.async(parse.json[City]) { request =>  
        weatherService.getWeatherForCity(request.body).value.map {  
            case Right(weather) => Ok(s"$weather")  
            case Left(error)    => InternalServerError(s"Error: $error")  
        }  
    }  
}
```

Funktionale Fehlerbehandlung

Funktionale Fehlerbehandlung

ApplicationError-Hierarchie



- Domain-Fehler als **gemeinsame Typhierarchie** statt roher Strings:
- Vorteil: alle Fehler teilen einen Obertyp → erlaubt einheitliches Mapping auf HTTP

```
trait ApplicationError extends Throwable {  
  def message: String  
}
```

```
case class BadRequest(message: String) extends ApplicationError  
case class WeatherDataError(statusCode: Int, message: String) extends ApplicationError
```

Funktionale Fehlerbehandlung

EitherT als roter Faden



- Realer Code ist ``Future[Either[Error, A]]`` — verschachtelt und sperrig
- **EitherT[Future, ApplicationError, A]** flacht das ab (VL06):
 - for-Comprehensions über mehrere Schritte, bricht bei ``Left`` ab
 - ``.value`` gibt am Ende ``Future[Either[ApplicationError, A]]`` zurück
- Unterschiedliche Fehler-Subtypen vereinheitlichen mit ``leftWiden``:

```
weatherService.getWeatherForCity(city) // EitherT[F, WeatherDataError, _]  
  .leftWiden[ApplicationError]          // → EitherT[F, ApplicationError, _]
```

Übung

- Erstelle ein **todo-Feature** nach Vorbild des fertigen **weather-Features**
- 1. Projekt & weather-Referenz verstehen
- 2. todo-Package & Modell
- 3. domain: Service-Interface (@ImplementedBy)
- 4. infrastructure: InMemory-Repository
- 5. application: Read-Endpunkte
- 6. Create (POST)
- 7. Update & Delete
- 8. Validierung mit ApplicationError
- 9. WeatherService integrieren
- 10. Todo & Wetter kombinieren

- ``sbt run`` oder mit IntelliJ run script starten und das fertige Wetter-Feature testen:
 - ``requests/GetWeatherForCity.http`` in IntelliJ ausführen (``POST /v1/weather``)
- Das VL07 Request Projekt ausführen:
 - Sendet 10 Requests gleichzeitig
 - Was fällt in beiden Logs auf?
- Das **weather-Feature als Architektur-Vorlage** lesen:
 - application / domain / infrastructure und ihre Aufgaben
- Ziel: das Muster verstehen, das wir gleich für Todo nachbauen

- Weitere Package-Struktur anlegen: ``todo/{application, domain, infrastructure}``
- Modell ``Todo`` mit ``given Format`` (analog zu ``City``)

- ``TodoService``-Trait im domain-Layer, gebunden via ``@ImplementedBy``:
- ``TodoServiceImpl`` ruft das Repository (Schritt 4) und liefert ``EitherT``

- ``TodoRepository`-Trait` (``@ImplementedBy``) + InMemory-DB nutzen

- `TodoController` (application) wie der `WeatherController`:
- Routes ergänzen: `GET /v1/todos`, `GET /v1/todos/:id` . . .
- Service erweitern

- ``POST /v1/todos`` mit typsicherem ``parse.json[Todo]``:
- ``request.body`` ist dank ``given Format[Todo]`` direkt ein ``Todo``

- ``PUT /v1/todos/:id`` — bestehendes Todo ersetzen (z.B. als erledigt markieren)
- ``DELETE /v1/todos/:id`` — Todo entfernen, Erfolg → ``NoContent`` (204)
- Service und Repository um ``update`` und ``delete`` erweitern

- Im Service vor dem Speichern prüfen

```
def validate(todo: Todo): EitherT[Future, ApplicationError, Todo] =  
    EitherT.cond[Future](todo.title.nonEmpty, todo, BadRequest("Titel leer"))
```


- `TodoService` zusätzlich den fertigen `WeatherService` injizieren
- `Todo` um optionales Feld `city: Option[City]` erweitern

```
class TodoServiceImpl @Inject() (  
    repo: TodoRepository,  
    weatherService: WeatherService  
) (using ExecutionContext) extends TodoService
```

- Bei `GET /v1/todos/:id` mit gesetzter City das Wetter dazuladen:
- **Bonus:** Wetter für mehrere Cities parallel mit `cities.traverse(...)` (cats)

```
for {  
  todo    <- todoService.findById(id)  
  city    <- EitherT.fromOption[Future](todo.city, BadRequest("keine City"))  
  weather <- weatherService.getWeatherForCity(city)  
                .leftWiden[ApplicationError]    // WeatherDataError <: ApplicationError  
} yield TodoWithWeather(todo, weather)
```